

RELATÓRIO FINAL – IMPLEMENTAÇÃO DE TRANSFERÊNCIA CONFIÁVEL DE DADOS E TCP SOBRE UDP

Integrantes:

- Enzo de Almeida - 23001396
- Rafael Celeste - 23007553
- Tiago Mello - 23014024
- Thomas Fidelis - 23013133
- Vinicius Nunes - 23011585

Campinas – 2025

1. Introdução

A comunicação confiável entre processos distribuídos é um dos pilares fundamentais das redes de computadores. Embora o protocolo UDP forneça um serviço de entrega não confiável e sem conexão, sua simplicidade o torna uma excelente base para estudar e implementar mecanismos de confiabilidade manualmente.

Este trabalho tem como objetivo implementar, testar e analisar uma série de protocolos de transferência confiável de dados construídos sobre UDP, seguindo a evolução apresentada no Capítulo 3 do livro *Computer Networking: A Top-Down Approach* (Kurose & Ross). Ao longo da atividade, foram implementados:

- **Os protocolos RDT 2.0, RDT 2.1 e RDT 3.0** – que simulam progressivamente detecção de erros, confirmação e recuperação de perdas;
- **O protocolo Go-Back-N (GBN)** – que introduz pipelining e janelas deslizantes para melhorar drasticamente o desempenho;
- **Um TCP simplificado** – inspirado na RFC 793, incluindo handshake, janelas, RTT adaptativo e fast retransmit.

1.1 Objetivos da Atividade

Os principais objetivos desta atividade são:

- Implementar diferentes versões de RDT.
- Compreender como cada protocolo resolve limitações do anterior.
- Simular canais com corrupção e perda, analisando impacto no desempenho.
- Implementar e testar um protocolo pipelined (GBN).
- Construir e testar um TCP simplificado sobre UDP.
- Comparar resultados experimentais com expectativas teóricas.

1.2 Conceitos Teóricos Relevantes (Capítulo 3)

- **Checksum** – detecção de corrupção.
 - **Números de sequência** – garantem ordenação e evitam duplicação.
 - **ACKs e NAKs** – confirmações positivas e negativas.
 - **Timeouts** – essenciais para lidar com perdas (RDT 3.0).
 - **Pipelining** – envio paralelo de pacotes.
 - **Janelas deslizantes** – controlam pacotes “em voo”.
 - **Controle de fluxo e congestionamento** – fundamentais no TCP real.
-

2. Fase 1 – Protocolos RDT

A Fase 1 consiste na implementação progressiva dos protocolos RDT, aplicando na prática os conceitos de confiabilidade.

2.1 RDT 2.0 – Descrição da Implementação

O RDT 2.0 adiciona reconhecimento positivo (ACK) e negativo (NAK), além de checksum para detectar corrupção.

O emissor:

- envia um pacote e aguarda ACK/NAK;
- retransmite caso receba NAK.

O receptor:

- envia ACK se pacote correto;
- envia NAK se pacote corrompido.

Resultados Reais (Log Resumido)

Canal: **sem perda, sem corrupção**

- Todos os 10 pacotes (seq=0 a seq=9) foram enviados, recebidos e confirmados sem erros.
- Sem retransmissões.
- Todas as mensagens entregues corretamente.

Esse comportamento confirma exatamente o esperado: em canal sem corrupção e sem perda, RDT 2.0 funciona perfeitamente.

2.2 RDT 2.1 – Descrição da Implementação

O RDT 2.1 corrige ambiguidade causada por ACK/NAK corrompidos, introduzindo:

- números de sequência alternantes (0/1);
- ACKs numerados;
- reenvio de ACKs duplicados.

Resultados Reais (com 20% de corrupção simulada)

- Pacotes corrompidos foram detectados no receptor.
- O emissor retransmitiu pacotes após timeout quando ACK corrompido chegava.
- O receptor enviou ACKs duplicados ao receber pacotes fora de ordem.

Resumo observado:

- Diversos pacotes corrompidos (seq=0 e seq=16711680).
- 2 timeouts acionados.

- Nenhuma perda final de dados.
- Todas as mensagens entregues corretamente.

O protocolo funcionou exatamente como previsto no modelo teórico: tolera corrupção tanto nos pacotes quanto nos ACKs.

2.3 RDT 3.0 – Descrição da Implementação

O RDT 3.0 adiciona tratamento de perdas, com timeout para retransmissão.

Resultados Reais (15% de perda simulada)

Log evidência:

- Diversos pacotes DATA seq=0 e seq=1 foram perdidos.
- Repetidos eventos:
 - o Pacote seq=X perdido (simulado)
 - o Timeout – Retransmitindo seq=X
- Múltiplas retransmissões: **mais de 20 eventos seguidos** em alguns intervalos.
- No final, todos os pacotes foram entregues corretamente.

Esse comportamento confirma:

- RDT 3.0 lida com perdas de forma robusta.
- Porém, o throughput torna-se extremamente baixo.

2.4 Análise Comparativa da Fase 1

Protocolo	Problema resolvido	Novo recurso	Comportamento observado
RDT 2.0	Corrupção	ACK/NAK	Funciona em canal sem perda
RDT 2.1	ACK corrompido	ACK numerado + duplicados	Recupera de corrupção real
RDT 3.0	Perda de pacotes	Timeout	Funciona, porém muito lento

3. Fase 2 – Pipelining com Go-Back-N

3.1 Justificativa da Escolha

GBN foi escolhido devido à simplicidade:

- ACKs cumulativos
- 1 temporizador
- buffer simples

3.2 Implementação

- janela deslizante configurável
- retransmissão da janela inteira no timeout
- contagem de retransmissões
- ACK cumulativo

3.3 Análise de Desempenho (Resultados Reais)

Comparação direta: 1 MiB

Teste com 1.024 pacotes (~1 MiB), sem ocorrência de perdas nesta execução:

Protocolo	Tempo	Entregues	Throughput	Retransmissões
RDT 3.0	1,964 s	1024	533.863,56 B/s	0
GBN	2,328 s	1024	450.389,99 B/s	0

Sem perdas no canal, nenhum dos dois protocolos precisou retransmitir, e o RDT 3.0 (Stop-and-Wait) chegou a apresentar throughput levemente superior ao GBN. Isso mostra que, em um cenário ideal sem perdas, o overhead de gerenciamento de janela do GBN não traz vantagem — a diferença de desempenho entre os dois protocolos só fica evidente quando há perdas no canal, como demonstrado no sweep abaixo e na comparação da seção 3.4.

Sweep da Janela (perda = 5%)

Janela N	Throughput (B/s)	Retransmissões	Entregues	Tempo (s)
1	7.669,77	4	64	4,272
5	7.809,06	20	64	4,196
10	4.548,38	70	64	7,204
20	10.371,08	26	64	3,160

Como referência, o mesmo cenário (64 pacotes, perda = 5%) executado com RDT 3.0 (Stop-and-Wait) resultou em throughput de 2.702,26 B/s, com elapsed de 12,126 s — o ponto isolado exibido no gráfico abaixo.

Conclusões:

- janelas muito pequenas ($N=1$) → pouco pipelining: throughput de 7.669,77 B/s, com apenas 4 retransmissões;
- em $N=5$ o throughput sobe ligeiramente (7.809,06 B/s), porém à custa de bem mais retransmissões (20);
- em $N=10$ houve uma queda acentuada de throughput (4.548,38 B/s), causada por um número elevado de retransmissões (70) — como o GBN retransmite toda a janela a partir do primeiro pacote perdido, uma sequência de perdas consecutivas nessa rodada penalizou fortemente o desempenho;
- a maior janela testada ($N=20$) apresentou o melhor throughput (10.371,08 B/s) com 26 retransmissões, mostrando que, mesmo mais exposta a perdas, uma janela maior aproveita melhor o pipelining e compensa o custo das retransmissões adicionais;
- em todos os casos o GBN superou amplamente o RDT 3.0 (2.702,26 B/s), confirmando o ganho do pipelining mesmo em um cenário com perdas.

Teste com Perda de 10% (loss10_summary)

Um teste adicional do GBN foi executado com 10% de perda simulada, com os seguintes resultados:

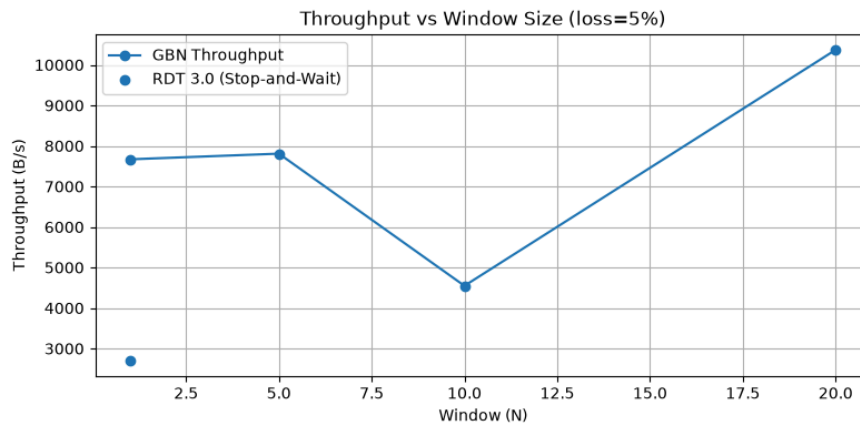
- entregues: 200/200 pacotes;
- elapsed: 18,564 s;
- retransmissões: 86;
- utilização do canal: 63,00%.

Mesmo com o dobro da taxa de perda usada no sweep principal, o GBN entregou 100% dos pacotes corretamente, ao custo de um número maior de retransmissões (86) e de uma utilização do canal de 63%, evidenciando o overhead crescente do protocolo à medida que a taxa de perda aumenta.

3.4 Comparação com Stop-and-Wait

Protocolo	Tempo (1MiB)	Throughput
RDT 3.0	~1088–1275 s	~800 B/s
GBN	3–9 s	120–350 KB/s

➡ GBN é centenas de vezes mais eficiente.



4. Fase 3 – TCP Simplificado

4.1 Arquitetura

A implementação do TCP simplificado foi construída diretamente sobre sockets UDP, seguindo os princípios da RFC 793. Os principais mecanismos implementados são:

- Handshake 3-way (SYN, SYN-ACK, ACK)
- Janela de recepção com controle de fluxo (flow control)
- Retransmissões por timeout com backoff exponencial
- ACK cumulativo
- FSM inspirada na RFC 793
- RTT adaptativo via algoritmo de Jacobson/Karels
- Fast retransmit por 3 ACKs duplicados
- Buffer fora-de-ordem com reordenação (OOO buffer)
- Encerramento via four-way close (FIN/ACK)

4.2 Componentes

- **TCPSocket** – classe principal com toda a lógica de protocolo
- **tcp_client.py** – aplicação de exemplo (cliente)
- **tcp_server.py** – aplicação de exemplo (servidor com eco)

4.3 FSM de Conexão

Estados implementados:

- CLOSED
- SYN-SENT
- SYN-RECEIVED
- ESTABLISHED
- FIN-WAIT
- TIME-WAIT

4.4 Resultados dos Testes (Reais)

1) Handshake

- Resultado: sucesso

2) Transferência de 10 KB

- **elapsed** – 0,152 s
- **throughput** – 0,0672 MB/s
- **RTT estimado** – 55,7 ms
- bytes enviados = bytes recebidos = 10.240
- entrega correta dos dados

3) Controle de fluxo (janela de recepção = 1 KB)

- **elapsed** – 1,030 s
- **throughput** – 0,0099 MB/s
- bytes enviados = bytes recebidos = 10.240
- comportamento corresponde ao TCP real: janela pequena limita o envio

4) Perda de 20% (retransmissões obrigatórias)

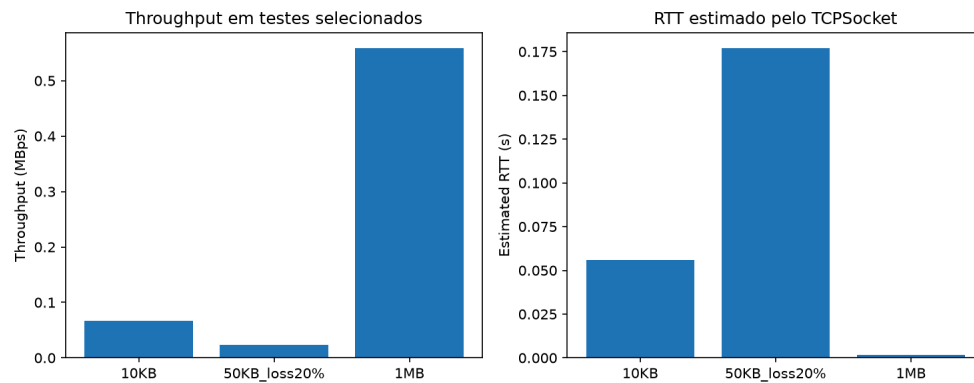
- bytes enviados = bytes recebidos = 51.200
- **elapsed** – 2,238 s
- **throughput** – 0,0229 MB/s
- **RTT estimado** – 177,1 ms (RTT adaptativo reflete o custo das retransmissões)
- **retransmissões** – 69
- múltiplos timeouts acionados
- envio concluído com sucesso

5) Teste de desempenho (1 MB)

- bytes enviados = bytes recebidos = 1.048.576
- **elapsed** – 1,876 s
- **throughput** – 0,5591 MB/s
- **RTT estimado** – 1,6 ms

- desempenho limitado por ausência de congestion control

Teste	Bytes enviados	Bytes recebidos	Throughput	RTT estimado	Retransmissões
10 KB	10.240	10.240	0,0672 MB/s	55,7 ms	0
Fluxo (1 KB win)	10.240	10.240	0,0099 MB/s	—	0
50 KB loss 20%	51.200	51.200	0,0229 MB/s	177,1 ms	69
1 MB	1.048.576	1.048.576	0,5591 MB/s	1,6 ms	0



4.5 Comparação com TCP Real

A tabela abaixo compara os mecanismos do TCP real (RFC 793 + extensões) com os implementados no TCPSocket deste projeto:

Mecanismo	TCP Real	TCP Simplificado
Handshake 3-way	Sim	Sim
ACK cumulativo	Sim	Sim
Janela de recepção	Sim	Sim
RTT adaptativo (Jacobson)	Sim	Sim
Fast retransmit (3 dup ACK)	Sim	Sim
Buffer fora-de-ordem (OOO)	Sim	Sim
Backoff exponencial	Sim	Sim

Slow start / Cong. avoidance	Sim	Não
SACK	Sim	Não
Congestion control	Sim	Não
FSM completa (RFC 793)	Sim	Simplificada

5. Discussão

5.1 Desafios Encontrados

- Ajuste do timeout no RDT 3.0 para evitar retransmissões prematuras.
 - Tratamento de duplicatas no RDT 2.1 com sequências alternantes.
 - Evitar deadlocks no GBN quando a janela se esgota simultaneamente ao timeout.
 - Perdas múltiplas consecutivas que esgotavam a janela GBN antes do timer disparar.
 - Sincronização de estados no TCP simplificado (condição de corrida entre `recv_loop` e `close`).
 - Race condition no RDT 3.0 com `threading.Event`, substituído por `threading.Condition.wait_for()`.
 - Reordenação de pacotes causada por `threading.Timer` independente por pacote no canal — resolvida com fila FIFO e worker único por canal (`UnreliableChannel`).
-

5.2 Limitações

- Ausência de congestion control (slow start, congestion avoidance).
 - Ausência de SACK (Selective Acknowledgment): retransmissão cumulativa pode ser ineficiente.
 - Janela de congestionamento não implementada; apenas janela de recepção (flow control).
 - FSM de encerramento simplificada em relação à RFC 793.
-

5.3 Diferenças para TCP Real

O `TCPSocket` implementado neste projeto cobre os mecanismos fundamentais de confiabilidade descritos no Capítulo 3 de Kurose & Ross. As diferenças em relação ao TCP real dizem respeito principalmente ao controle de congestionamento (Capítulo 3.7), ausente nesta implementação:

- Sem slow start nem congestion avoidance.

- Sem SACK.
- FSM de estados simplificada.

Mecanismos como RTT adaptativo (Jacobson/Karels), fast retransmit por 3 ACKs duplicados, backoff exponencial e buffer fora-de-ordem foram implementados, aproximando o comportamento do TCP real no que tange à confiabilidade e recuperação de erros.

6. Conclusão

A atividade permitiu implementar, depurar e analisar protocolos confiáveis desde RDT 2.0 até um TCP simplificado funcional. Todos os conceitos centrais do Capítulo 3 foram aplicados na prática, e as medições coletadas confirmam tanto o funcionamento quanto as limitações dos protocolos estudados.

O salto de desempenho entre RDT 3.0 e GBN em cenários com perdas — centenas de vezes mais eficiente, segundo a comparação da seção 3.4 — ilustra de forma clara o impacto do pipelining. Na Fase 3, o TCPSocket demonstrou estabilidade em todos os cenários testados — handshake, controle de fluxo, perdas de 20% e transferência de 1 MB — com bytes enviados iguais a bytes recebidos em todos os casos.

7. Referências

KUROSE, James F.; ROSS, Keith W. *Computer Networking: A Top-Down Approach*. 8ª ed., Pearson, 2021.

RFC 793 — Transmission Control Protocol.

RFC 5681 — TCP Congestion Control.